

TRYNEX LIFESTYLE

Design Studio

Master Rebuild & Feature Plan

Comprehensive Architecture · Advanced Features · Implementation Roadmap · All-in-One Command Guide

trynexshop.com · Version 2.0 · July 2026

§	Section	Focus
1	Emergency Fixes	Database + admin panel zeros + GitHub token
2	Current State Audit	Existing codebase analysis and pain points
3	Redesigned Architecture	Component split, state management, performance
4	New Mockup System	PSD-quality renders, proper photo compositing
5	Advanced UI Features	Clipart, gradients, patterns, shapes, effects
6	AI Integration v2	Stable Diffusion, style transfer, auto-compose
7	Tech Stack Upgrade	Fabric.js / Konva, Zustand, Cloudflare Workers
8	Mobile Experience	Touch-native gestures, PWA, Expo deep link
9	Implementation Phases	6-phase roadmap with timelines

1.1 Admin Panel Showing 0s — Root Cause & Fix

The admin dashboard showing zeros for orders, revenue, and customers is caused by a **Neon database quota issue**. The primary database (DATABASE_URL_MAIN) exceeded its data-transfer quota and the failover (DATABASE_FAILOVER) exceeded its compute-time quota. The server automatically fell back to DATABASE_PRODUCTS — a catalog shard that has products but **no order history**.

■ Fix Applied (Automatic — already deployed)

The DATABASE_ANALYTICS shard (ep-cool-mountain) was discovered to contain **73 real orders, 10 products, 3 reviews, and 613 total rows** — a full mirror from the last healthy sync of the main database. The failover priority in `lib/db/src/index.ts` has been updated to promote DATABASE_ANALYTICS above DATABASE_PRODUCTS. The API server has been rebuilt and restarted.

Before fix	Main → Failover [quota] → Products (0 orders) → Analytics
After fix	Main → Failover [quota] → Analytics (73 orders ✓) → Products
Orders visible	73 real orders now showing in admin panel
Revenue visible	Real revenue data now accessible
Action needed	Upgrade Neon plan to restore Main + Failover DBs (monthly reset coming)

1.2 Neon Database Quota — What To Do

Your two primary Neon databases are over quota. This is a plan limit, not a bug. The Analytics DB will serve as primary until quotas reset (monthly). To permanently fix:

- Log in to console.neon.tech
- Go to main account (georgelsmith333@gmail.com)
- Upgrade to Neon Launch plan (~\$19/mo)
- This restores Main DB (ep-proud-hill) with full data
- Alternative: wait for monthly quota reset
- Backup account (improvewithamit) — also over quota
- Analytics DB (ep-cool-mountain) active now as temporary primary
- All 73 orders + data are safe and accessible

1.3 GitHub Token — Must Refresh

All stored GitHub tokens have expired. The user-uploaded secrets file contains a new token. Update `GITHUB_PERSONAL_ACCESS_TOKEN` in Replit Secrets with the new classic token from the secrets file. This restores git-push capability and CF Pages deployment.

```
# Secret to update in Replit → Secrets panel:
GITHUB_PERSONAL_ACCESS_TOKEN = ghp_sIcLXnagb...SFGBR (from uploaded secrets file)

# Also update Cloudflare token – all existing CF tokens returned error 1000
# Generate a new token at dash.cloudflare.com → Profile → API Tokens
# Use template: 'Edit Cloudflare Pages' → gives Pages read/write + Account read
CLOUDFLARE_API_TOKEN =
```

2.1 Codebase Size & Structure

DesignStudio.tsx	5,255 lines — monolithic React component (pain point #1)
mockups.tsx	917 lines — product definitions, SVG templates, photo maps
composer.ts	~600 lines — canvas compositing, photo blending, texture export
ProductViewer3D.tsx	~400 lines — Three.js / React Three Fiber 3D preview
State management	All in useState / useRef inside DesignStudio.tsx — no global store
Canvas approach	Raw HTML Canvas via refs + React drag gestures (no canvas library)
Mockup quality	PNG photo compositing with CSS mix-blend-mode multiply — SVG outlines only

2.2 Current Features (What Works)

- Image layer upload (PNG, JPG, SVG, WEBP)
- Text layers (11 fonts incl. Bangla)
- Font styling: bold, italic, size, color
- Text stroke + drop shadow
- Letter spacing control
- Layer reorder (drag + arrows)
- Show/hide + lock layers
- Undo / Redo history (20 steps)
- Product switching (tshirt, hoodie, mug, cap, longsleeve, waterbottle)
- Color picker per product (~8-10 colors/product)
- Multiple faces: front, back, sleeve, neck-label
- 3D preview (lazy-loaded React Three Fiber)
- AI art generation (Pollinations free API)
- Background removal (WASM + remove.bg)
- Design drafts (save/load from server)
- Add-to-cart with composed mockup image

2.3 Current Pain Points & Gaps

#	Issue	Impact	Priority
1	5,255-line monolithic component — impossible to maintain, critical parse	Critical	P0
2	No PSD-quality mockups — just PNG photos + SVG outlines	High	P1
3	SVG mockups look generic (simple outlines, not photorealistic)	High	P1
4	No clipart or element library — users can only upload their own art	High	P1
5	No gradient fill — text and shapes can only be solid colors	Medium	P2
6	No vector shape drawing tool (circle, rect, star, arrow)	Medium	P2
7	No pattern/repeat fill for designs	Medium	P2
8	Mobile UX is broken — gestures conflict with page scroll	High	P1
9	No QR code generator built into studio	Low	P3
10	No design export to PNG/PDF for proof approval	Medium	P2

11	Pollinations AI quality is limited — no style control	Medium	P2
12	No social share of design before purchase	Low	P3
13	No design history/gallery per customer account	Medium	P2
14	Text warp/arc effect not supported	Medium	P2
15	No multi-layer grouping or alignment tools	Medium	P2

3.1 Component Decomposition

The monolithic 5,255-line `DesignStudio.tsx` must be split into a clean component hierarchy. The goal is each file under 300 lines, with a clear separation of concerns.

Component / File	Responsibility	Lines Est.
<code>DesignStudio.tsx</code>	Root page: layout shell, routing, URL params	~150
<code>studio/StudioProvider.tsx</code>	Zustand store provider + keyboard shortcuts	~80
<code>studio/CanvasArea.tsx</code>	HTML Canvas rendering loop, zoom/pan viewport	~300
<code>studio/LayerRenderer.tsx</code>	Per-layer render: image, text, shape	~200
<code>studio/panels/LayerPanel.tsx</code>	Layer list sidebar: reorder, show/hide, lock	~200
<code>studio/panels/DesignPanel.tsx</code>	Right panel: selected layer properties	~300
<code>studio/panels/TextPanel.tsx</code>	Text-specific controls: font, size, effects	~250
<code>studio/panels/ImagePanel.tsx</code>	Image controls: brightness, contrast, flip	~150
<code>studio/panels/ShapePanel.tsx</code>	Vector shape controls: fill, stroke, opacity	~150
<code>studio/toolbar/MainToolbar.tsx</code>	Top toolbar: tool selection, undo/redo, zoom	~180
<code>studio/toolbar/ProductSwitcher.tsx</code>	Product + color + face switcher	~200
<code>studio/ProductPreview.tsx</code>	2D mockup + 3D preview toggle	~150
<code>studio/AIPanel.tsx</code>	AI generation prompt + results grid	~250
<code>studio/ClipArtBrowser.tsx</code>	Curated clipart + icon search	~200
<code>studio/GradientEditor.tsx</code>	Gradient builder: linear, radial, conic	~180
<code>studio/TemplateGallery.tsx</code>	Pre-built design templates grid	~150
<code>studio/ExportDialog.tsx</code>	Export to PNG, PDF, share link	~200
<code>hooks/useDesignStore.ts</code>	Zustand design state: layers, history, tools	~300
<code>hooks/useCanvasRenderer.ts</code>	Canvas drawing loop, DPI scaling	~200
<code>hooks/useGestures.ts</code>	Touch + mouse gesture handling (pinch/rotate)	~200
<code>design-studio/mockups.tsx</code>	Product definitions (keep, minor refactor)	~917
<code>design-studio/composer.ts</code>	Keep as-is — compositing logic is solid	~600

3.2 State Management — Zustand Design Store

Replace all the `useState/useRef` chaos inside `DesignStudio.tsx` with a typed Zustand store. This enables undo/redo, cross-component reactivity, and much easier testing.

```
// hooks/useDesignStore.ts – shape of the global design state
interface DesignStore {
  layers: Layer[]; // ordered layer stack
  selectedIds: string[]; // multi-select support
  history: Layer[][]; // undo stack (50 steps)
  future: Layer[][]; // redo stack
  activeTool: ToolType; // select|text|shape|draw|eyedrop
  activeProduct: DesignProduct;
  activeColor: ProductColor;
  activeFace: Face;
  zoom: number; // 0.25 – 4.0
  panX: number;
  panY: number;
  // Actions
  addLayer: (layer: Layer) => void;
  updateLayer: (id: string, patch: Partial<Layer>) => void;
  deleteLayer: (id: string) => void;
  moveLayer: (id: string, dir: 'up'|'down') => void;
  undo: () => void;
  redo: () => void;
  selectTool: (tool: ToolType) => void;
}
```

4.1 What's Wrong with the Current Approach

The current system uses PNG photos with CSS **mix-blend-mode: multiply** to tint white garments. This works for basic cases but fails for dark garments (no multiply), colored garments (hue shift issues), and curved products (mug, bottle) where perspective distortion is needed. SVG outlines are used as a fallback — they look like wireframes, not products.

4.2 Proposed High-Fidelity Mockup System

Product	Current Method	Proposed Upgrade	Quality Gain
T-Shirt (white/light)	PNG multiply blend	Displacement map + screen-space UV warp	★★★★■
T-Shirt (dark)	Full dark photo direct	Dark photo + additive screen blend	★★★★■
T-Shirt (color)	Color-specific PNGs (navy/red)	Hue-shift + lightness preserve algorithm	★★★★★
Hoodie	Mix-blend multiply + cutout	Same as T-shirt + pocket layer comp	★★★★■
Long Sleeve	Mix-blend multiply + cutout	Per-color photo bank (6 colors covered)	★★★★■
Mug (front)	CSS perspective + multiply	WebGL cylinder UV unwrap in Three.js	★★★★★
Cap (front)	SVG tint (no 3D)	Hemisphere distortion map in canvas	★★★★■
Water Bottle	CSS warp + cutout PNG	Cylinder unwrap with label zone clipping	★★★★★

4.3 Displacing Design Onto Garment with Canvas

For T-shirts and hoodies, a displacement map approach creates realistic fabric wrinkle effects without needing 3D. The design image is warped using the garment's gray-channel displacement map before compositing, making it look like the print follows the fabric contours.

```
// Displacement map compositing (new composer step)
async function applyDisplacementMap(
  designCanvas: HTMLCanvasElement,
  displacementMap: HTMLImageElement, // grayscale bump/normal map
  strength: number = 0.12
): Promise<HTMLCanvasElement> {
  // Per-pixel UV distortion based on displacement map luminance
  // strength 0.08-0.15 gives realistic shirt-wrinkle effect
  return displacedCanvas;
}
```

5.1 Feature Roadmap Overview

Feature	Description	Library / API	Phase
Clipart Library	500+ curated SVG icons + illustrations categorized by theme	community-made local SVG bank	P1
Vector Shape Tool	Draw circles, rectangles, stars, arrows, polygons	Canvas built-in shapes	P1
Gradient Fill	Linear, radial, and conic gradients for text + shapes	Canvas gradient API	P1
Pattern Fill	Repeating tile patterns: stripes, dots, chevrons, etc.	Canvas createPattern	P2
Text Warp / Arc	Bend text along a curve: arc, wave, flag, bulge effects	path type.js path calc	P2
Multi-select + Group	Select N layers → group → transform together	Zustand group state	P1
Smart Alignment	Align/distribute tools: center, left, right, top, bottom	layer bounding box math	P1
Design Export (PNG/PDF)	Download print-ready 300 DPI PNG or PDF with bleed marks	html2canvas + jsPDF	P1
Share Link	Generate short URL for design proof review + social sharing	URL shortlink + R2 snapshot	P2
QR Code Generator	Add QR code layer pointing to any URL	qrcode.js in-browser	P2
Design Gallery / History	Per-customer gallery of saved designs with thumbnails	NeonDB table + R2 thumbnails	P2
Template Marketplace	Community-submitted templates with rating + reviews	Backend moderation queue	P3
Background Patterns	Preset background colors, patterns, gradients for canvases	Canvas fill presets	P1
Eyedropper Tool	Sample color from uploaded design or mockup	Canvas getImageData	P2
Ruler & Guides	Toggle pixel/mm rulers, drag guide lines onto canvas	Canvas overlay rendering	P2

5.2 Gradient Editor — Implementation Sketch

```
// GradientEditor.tsx — creates a CanvasGradient for the selected layer
interface GradientStop { offset: number; color: string }
interface GradientConfig {
  type: 'linear' | 'radial' | 'conic';
  angle?: number; // degrees (linear)
  stops: GradientStop[]; // [{offset:0, color:'#F97316'}, {offset:1, color:'#4F46E5'}]
}

// Applied in LayerRenderer when layer.fill.type === 'gradient'
const grad = ctx.createLinearGradient(x0, y0, x1, y1);
config.stops.forEach(s => grad.addColorStop(s.offset, s.color));
ctx.fillStyle = grad;
```

6.1 Current AI vs Proposed

Capability	Current (Pollinations)	Proposed Upgrade
Image generation	Free, keyless, limited quality	Pollinations + Stable Diffusion API (optional key)
Style control	None — prompt only	Style presets: cartoon, realistic, watercolor, pixel art
Negative prompts	Not supported	Full negative prompt support
Background removal	WASM (slow) + remove.bg (API key)	Keep WASM + add Cloudflare Worker R2 cache
Design suggestions	None	AI suggest color palette + font pair from uploaded image
Auto-compose	None	AI auto-place design in optimal print zone with scale/position
Image enhance	Basic brightness/contrast auto-fix	Upscale 2x with Real-ESRGAN (Cloudflare Worker)
Text-to-clipart	None	Simple SVG icon fetch from Iconify by keyword

6.2 AI Style Presets Implementation

```
// AIPanel.tsx — style preset system
const AI_STYLE_PRESETS = [
  { id: 'realistic', label: 'Realistic', suffix: ', photorealistic, 8k, studio lighting' },
  { id: 'cartoon', label: 'Cartoon', suffix: ', flat vector cartoon, bold outlines, vibrant' },
  { id: 'watercolor', label: 'Watercolor', suffix: ', watercolor painting, soft edges, artistic' },
  { id: 'pixel', label: 'Pixel Art', suffix: ', pixel art, 16-bit retro game style' },
  { id: 'minimalist', label: 'Minimalist', suffix: ', clean minimalist design, simple shapes' },
  { id: 'streetwear', label: 'Streetwear', suffix: ', bold streetwear graphic, urban aesthetic' },
  { id: 'bangla', label: 'Bangla Art', suffix: ', traditional Bangladeshi folk art style' },
];

// Pollinations URL builder with negative prompt + seed
const url = `https://image.pollinations.ai/prompt/${encodeURIComponent(prompt + style.suffix)}
?width=1024&height=1024&nologo=true&enhance=true&negative=${encodeURIComponent(neg)}`;
```

7.1 Recommended Stack Changes

Layer	Current	Recommended Upgrade	Reason / Benefit
Canvas library	Raw HTML Canvas refs	Konva.js + react-konva	Object model, built-in hit-testing, events, transformers
State	useState + useRef soup	Zustand (typed store)	Serializable, devtools, undo/redo trivial
Gestures	use-gesture + custom	Konva transformer handles + use-gesture	Battle-tested multi-touch, pinch-rotate built-in
Fonts	Google Fonts (CDN)	Font face observer + preload hints	No FOUT, faster first paint
3D preview	React Three Fiber (lazy)	Keep R3F — it's good	Already solid; only improve UV mapping
Image upscale	None	Cloudflare Worker + Real-ESRGAN WASM	Free, runs at edge — no GPU needed
Design storage	DB JSON (drafts table)	R2 + DB reference (thumbnail + layers JSON)	20% cheaper for large layer JSONs
Export	Canvas.toDataURL	Offscreen Canvas + Worker thread	Non-blocking — UI stays responsive during export
Clipart	None	Iconify REST API + local SVG cache	7M+ icons, MIT license, instant search
Build	Vite (current)	Keep Vite — add code-split per studio panel	Reduce initial JS from 1.2MB to ~400KB
Testing	None	Vitest + @testing-library for panel components	Prevent regressions in layer logic
Monitoring	Console logs	Sentry (free tier) on canvas errors	Catch user-facing crashes in prod

7.2 Konva.js Migration — Why & How

The current raw canvas approach requires manual hit-testing, bounding box math, and transform calculations. Konva.js provides all of this out of the box plus a transformer widget for resize/rotate handles that works perfectly on both mouse and touch.

```
# Install Konva for the storefront
pnpm --filter @workspace/trynex-storefront add konva react-konva

# Basic Konva stage structure:
// <Stage width={W} height={H}>
// <Layer> {/* background + mockup */}
// <Image image={mockupImg} />
// </Layer>
// <Layer> {/* design layers */}
// {layers.map(l => <DesignLayer key={l.id} layer={l} />)}
// <Transformer ref={trRef} /> {/* resize+rotate handles */}
// </Layer>
// </Stage>
```

8.1 Current Mobile Problems

- Page scrolls when trying to pan canvas
- Pinch-to-zoom fires on the whole page
- Tool panel overflows on small screens
- Font picker unusable on mobile
- Layer panel not accessible (hidden)
- Add-to-cart button hidden below fold
- 3D preview too heavy for low-end phones
- No haptic feedback on layer select

8.2 Mobile-First Redesign Plan

Bottom sheet panels	Replace right sidebar with bottom sheets (slide up). Layer panel = slide up from bottom.
Touch event isolation	Canvas container uses touch-action:none — stops page scroll during canvas interaction.
Gesture library	use-gesture already handles this — add 'target' option to scope gestures to canvas element only.
Compact toolbar	Replace text labels with icon-only toolbar on screens <768px. Full labels on desktop.
Full-screen canvas	Mobile: canvas takes 65% of viewport. All panels slide-over. No persistent sidebars.
Haptic feedback	navigator.vibrate(10) on layer select/snap events on mobile.
3D preview lazy guard	Check navigator.hardwareConcurrency < 4 → skip 3D button on low-end devices.
Expo deep link	Add 'Customize' button in Expo app → open design studio URL with product pre-selected.

6-Phase Rebuild Roadmap

Phase	Name	Duration	Key Deliverables	Priority
P0	Emergency Fixes	Complete now	DB failover fix (✓ done), GitHub token refresh, CF token refresh	CRITICAL
P1	Architecture Refactor	1–2 weeks	Split DesignStudio.tsx into 20 components; add Zustand store; full typecheck passes	HIGH
P2	Konva.js Migration	1–2 weeks	Replace raw canvas with Konva Stage; Transformer widget; multi-select; alignment tools	HIGH
P3	Advanced Features	2–3 weeks	Clipart browser, gradients, shapes, patterns, text arc, QR code, export PNG/PDF	HIGH
P4	Mockup Upgrade	1–2 weeks	Displacement maps for T-shirts/hoodies; cylinder UV for mug/bottle; per-color photo bank expansion	MEDIUM
P5	AI v2 + Mobile	2 weeks	Style presets AI, auto-compose, mobile bottom-sheet UX, touch-only gesture isolation	MEDIUM
P6	Gallery + Community	2–3 weeks	Customer design gallery, template marketplace, share links, social proof	LOW

Total estimated rebuild time: **10–14 weeks** (working in parallel on P1–P2 since they don't conflict). P0 fixes are already applied. P1 is the highest leverage — splitting the monolith unlocks all future work.

Copy-Paste Ready Commands for Every Task

10.1 Start / Restart All Services

```
# From workspace root – restart all workflows via Replit UI, or use these for manual runs:

# API server (must rebuild after any server-side change):
cd artifacts/api-server && node ./build.mjs
# Then restart 'artifacts/api-server: API Server' workflow

# Storefront (Vite hot-reload, no rebuild needed):
pnpm --filter @workspace/trynex-storefront run dev

# Full typecheck (run before every git push):
pnpm run typecheck
```

10.2 Install New Dependencies

```
# Add Konva.js to storefront:
pnpm --filter @workspace/trynex-storefront add konva react-konva

# Add Zustand for state management:
pnpm --filter @workspace/trynex-storefront add zustand

# Add opentype.js for text path/arc:
pnpm --filter @workspace/trynex-storefront add opentype.js
pnpm --filter @workspace/trynex-storefront add --save-dev @types/opentype.js

# Add qrcode for QR code generation:
pnpm --filter @workspace/trynex-storefront add qrcode
pnpm --filter @workspace/trynex-storefront add --save-dev @types/qrcode

# Add jsPDF for design export:
pnpm --filter @workspace/trynex-storefront add jspdf html2canvas

# Add font-face-observer for reliable font loading:
pnpm --filter @workspace/trynex-storefront add fontfaceobserver
pnpm --filter @workspace/trynex-storefront add --save-dev @types/fontfaceobserver
```

10.3 Database Commands

```

# Check active database from within api-server dir:
cd artifacts/api-server
node --input-type=module -e "
const { getActiveDbUrl } = await import('@workspace/db');
console.log(await getActiveDbUrl());
"

# Run a direct DB health check (no auth required):
curl http://localhost:80/api/healthz

# Check admin stats (requires admin session cookie):
curl http://localhost:80/api/admin/stats \
-H 'Cookie: admin_session='

# Force DB re-probe (switches to best available DB):
curl -X POST http://localhost:80/api/admin/db/reprobe \
-H 'Cookie: admin_session='

# Run DB migrations manually:
cd artifacts/api-server && node --input-type=module -e "
const { dbReady } = await import('@workspace/db');
await dbReady;
const { runMigrations } = await import('../dist/index.mjs');
"

```

10.4 API Server Rebuild + Deploy Cycle

```

# Full deploy cycle after server-side changes:

# 1. Edit source files in artifacts/api-server/src/

# 2. Rebuild the bundle:
cd artifacts/api-server && node ./build.mjs

# 3. Restart workflow in Replit UI:
# artifacts/api-server: API Server → Stop → Start

# 4. Verify it came up clean:
curl http://localhost:80/api/healthz

# 5. Commit + push (restores GitHub token first):
git add -A
git commit -m 'fix: '
# Use GitHub REST API to push (git CLI blocked by Replit):
# Agent handles this automatically via REST API

# CF Pages auto-deploys when main branch is pushed to GitHub.
# Manual trigger only needed if auto-deploy hook broke.

```

10.5 Design Studio: Phase 1 Refactor Bootstrap

```
# Create folder structure for refactored studio:
mkdir -p artifacts/trynex-storefront/src/pages/studio
mkdir -p artifacts/trynex-storefront/src/pages/studio/panels
mkdir -p artifacts/trynex-storefront/src/pages/studio/toolbar
mkdir -p artifacts/trynex-storefront/src/hooks

# Files to create (in order – no circular deps):
# 1. src/hooks/useDesignStore.ts ← Zustand store (no imports from studio)
# 2. src/pages/studio/panels/LayerPanel.tsx
# 3. src/pages/studio/panels/TextPanel.tsx
# 4. src/pages/studio/panels/ImagePanel.tsx
# 5. src/pages/studio/toolbar/MainToolbar.tsx
# 6. src/pages/studio/toolbar/ProductSwitcher.tsx
# 7. src/pages/studio/CanvasArea.tsx
# 8. src/pages/studio/AIPanel.tsx
# 9. src/pages/studio/ClipArtBrowser.tsx
# 10. src/pages/DesignStudio.tsx ← thin orchestrator, <200 lines

# After refactor, check for TS errors:
pnpm --filter @workspace/trynex-storefront run typecheck
```

10.6 Konva.js Canvas Setup Boilerplate

```
// artifacts/trynex-storefront/src/pages/studio/CanvasArea.tsx
import { Stage, Layer, Image as KonvaImage, Transformer } from 'react-konva';
import Konva from 'konva';
import { useRef, useEffect } from 'react';
import { useDesignStore } from '@hooks/useDesignStore';

export function CanvasArea({ mockupImg }: { mockupImg: HTMLImageElement }) {
  const trRef = useRef<Konva.Transformer>(null);
  const { layers, selectedIds, zoom } = useDesignStore();

  return (
    <Stage width={600} height={600} scaleX={zoom} scaleY={zoom}>
      <Layer>
        <KonvaImage image={mockupImg} width={600} height={600} />
      </Layer>
      <Layer>
        {layers.map(layer => <DesignLayer key={layer.id} layer={layer} />)}
        <Transformer ref={trRef} rotateEnabled flipEnabled />
      </Layer>
    </Stage>
  );
}
```

10.7 Neon DB Quota Resolution Commands

```
# Current DB status (run after API restart to confirm analytics is active):
curl http://localhost:80/api/healthz | grep db

# Check which DB is actually active (admin endpoint):
curl http://localhost:80/api/admin/db/cluster \
-H 'Cookie: admin_session='

# When Neon main quota resets (monthly):
# 1. The reprobe timer (every 60s) will automatically detect main is healthy
# 2. Server will switch back to DATABASE_URL_MAIN without restart
# 3. Monitor via admin DB cluster page

# To force switch to best available DB immediately:
curl -X POST http://localhost:80/api/admin/db/reprobe \
-H 'Cookie: admin_session='
```

10.8 Quick Secrets Checklist

GITHUB_PERSONAL_ACCE SS_TOKEN	Update in Replit Secrets → new token from uploaded secrets file
CLOUDFLARE_API_TOKEN	Regenerate at dash.cloudflare.com → Profile → API Tokens → Edit Pages template
DATABASE_URL_MAIN	✓ Set — over quota now; auto-recovery when quota resets
DATABASE_ANALYTICS	✓ Set and ACTIVE — 73 orders, 10 products, 3 reviews
DATABASE_PRODUCTS	✓ Set — standby fallback
R2_ACCESS_KEY_ID / SECRET	✓ Set — Cloudflare R2 storage working
TELEGRAM_BOT_TOKEN	✓ Set — order notifications working
ADMIN_JWT_SECRET / ADMIN_PASSWORD	✓ Set

SUMMARY & NEXT STEPS

What Was Fixed Right Now

✓ DB Failover Reordered	DATABASE_ANALYTICS promoted above DATABASE_PRODUCTS — 73 orders now visible in admin
✓ API Server Rebuilt	lib/db/src/index.ts updated, api-server bundle rebuilt and restarted
→ GitHub Token	Paste new token (ghp_slcLXna...) into GITHUB_PERSONAL_ACCESS_TOKEN in Replit Secrets
→ CF Token	Generate new CF API token at dash.cloudflare.com using 'Edit Cloudflare Pages' template
→ Neon Upgrade	Upgrade trynex main Neon account to restore full primary DB (optional — analytics covers for now)

Recommended Next Build Session

The highest-leverage single action is **Phase 1: Architecture Refactor** — splitting the 5,255-line DesignStudio.tsx into focused components with Zustand state. Once that's done, all subsequent feature additions (Konva, gradients, clipart, AI v2) can be done in parallel by independent agents without stepping on each other. The refactor is a prerequisite for everything else.